OUF: Oblivious Universal Function with domain specific optimizations

Victor Delfour
Université du Quebec à Montréal (UQAM)
Montréal, Canada
Email: delfour.victor@courrier.uqam.ca

Marc-Olivier Killijian
Université du Quebec à Montréal (UQAM)
Montréal, Canada
Email: killijian.marc-olivier.2@uqam.ca

***Abstract*—The growing need for secure computation has spurred interest in cryptographic techniques that operate on encrypted data without revealing its content. Fully Homomorphic Encryption (FHE), especially LWE-based schemes, enables such processing while preserving confidentiality. Decentralized computing offers scalable resources without requiring in-house servers, but it relies heavily on the confidentiality guarantees of underlying schemes. While many existing protocols successfully protect input privacy, function confidentiality remains a largely overlooked but crucial aspect of secure delegated computation.**

In this work, we present a novel Oblivious Universal Function (OUF) scheme that enables a client to outsource computation of an arbitrary function to an untrusted server while hiding both the input data and the function being applied. Our construction leverages LWE-based FHE and a virtual-tape evaluation model to support composable, non-interactive, and reusable function execution. Crucially, the server remains oblivious not only to the encrypted inputs but also to the structure, type, and identity of the function it is executing. OUF thus bridges the gap between theoretical privacy guarantees and practical secure computation in decentralized environments.

I. Introduction

Cryptographic techniques that support computation over encrypted data are central to privacy-preserving computing. Among them, Fully Homomorphic Encryption (FHE), particularly schemes based on the Learning with Errors (LWE) problem, enables secure function evaluation without compromising input confidentiality. These capabilities are especially valuable in decentralized computing, where organizations outsource computations to untrusted servers.

While most efforts focus on protecting inputs and outputs, the confidentiality of the function itself remains an often-overlooked yet critical aspect. In many applications, the function encodes proprietary or sensitive logic whose exposure can reveal business strategies or user-specific behaviors.

For example, in finance, the structure of a trading algorithm may disclose asset selection criteria, risk models, or reaction to market conditions, even if inputs and outputs are encrypted. Similarly, in healthcare or security-sensitive contexts, knowing the function may leak organizational intent or user profiles. Function leakage thus undermines both privacy and competitiveness.

Full confidentiality, covering inputs, outputs, and function logic, requires oblivious computation. Oblivious Turing Machines (OTMs) are the only known construction offering all three guarantees. However, their practical limitations are severe: they are slow, constrained by cryptographic parameters, and limited in supported functionality.

This motivates the need for more efficient alternatives. In this work, we introduce a new scheme that retains the privacy guarantees of OTMs while significantly improving performance, scalability, and ease of use.

A. Our contribution

This paper introduces a scheme allowing to non-interactively obliviously compute any function in a client-server context where the client possess a private function and private data and wants the server to evaluate the function on its data without learning any information on the function or data besides some upper bounds as the size of the function or data and the time of execution. This type of scheme was introduced with Oblivious Turing Machines but these constructions are very slow and very constrained by cryptographic parameters with small data size and very limited functions in practice. Our contribution is to introduce a scheme with the same properties while more efficient, easier to use and less constrained by cryptographic parameters.

In this work, we introduce an oblivious computing scheme that aims to natively integrate function confidentiality, enabling complex and versatile computations with strong privacy guarantees. Our design extends current frameworks by supporting scalable, non-interactive evaluation of arbitrary functions.

At its core, the scheme supports the computation of any function, regardless of its complexity, while ensuring obliviousness to both external observers and the computing party. We formally prove its Turing completeness, demonstrating its theoretical ability to simulate any computation achievable by a Turing machine.

A key contribution is the scheme's support for oblivious evaluation of bracketed expressions over arbitrary bivariate functions. This includes operations such as comparisons, conditionals, and more advanced mathematical functions, such as distance computations in machine learning models or risk scoring in financial applications. By directly handling bivariate relationships, the scheme preserves complex data

dependencies such as correlations between variables or evolving trends without leaking structure to the computing party. This provides a more tractable alternative to traditional Turing machine-based approaches, reducing computational overhead while retaining flexibility and privacy. This ability to hide computation logic is crucial for privacy in real-world scenarios

The scheme is non-interactive, requiring only a single round of communication between client and server independently of the number of operations or function complexity. This minimizes client-side computation and bandwidth usage, making it particularly suitable for decentralized environments. Moreover, inputs and functions can be reused indefinitely, further reducing communication costs in repeated evaluations. We validate our approach with a proof-of-concept implementation.

B. Related Work

Our work falls within the domain of Secure Function Evaluation (SFE), a field that has traditionally emphasized protecting the confidentiality of participants' inputs, while comparatively little attention has been given to preserving the secrecy of the function itself. Preserving function privacy is particularly important in settings where the computation logic itself can reveal sensitive information. This section reviews various existing approaches to secure computation, with particular emphasis on their treatment of input, function, and output privacy, and highlights how our work fits within and extends this landscape. We put some emphasis on how most of these constructions do not achieve the same goal as ours, which is protecting both the function, data and result privacy from a server non-interactively.

Homomorphic Encryption (HE) allows clients to delegate computations to untrusted servers without revealing their inputs. However, the evaluated function remains visible to the server, limiting privacy. To address this, universal homomorphic functions aim to treat the function itself as encrypted data, allowing secure evaluation of private functions. Our scheme falls into this family of techniques, providing a new approach to achieve both input and function confidentiality in a decentralized setting.

Private Function Evaluation (PFE) extends HE by allowing one party to evaluate a private function on another party's private inputs. While effective in the semi-honest adversary model, especially for boolean circuits [1]–[4], PFE typically targets two-party settings and does not naturally extend to decentralized environments where the client owns both the function and the inputs, as is the case in our model.

Function-Hiding Functional Encryption (FE) achieves stronger theoretical guarantees by enabling the evaluation of encrypted functions on encrypted inputs without revealing either [5], [6]. However, FE differs substantially from oblivious function evaluation: in FE, the server learns the result, whereas in our setting, the computing party learns nothing at all. Furthermore, FE schemes generally do not support function or input reusability, which limits their practical applicability to many privacy-preserving applications.

Universal Circuits (UCs), proposed by Valiant [7], offer another approach by enabling the simulation of any circuit of a given size while concealing its structure. Schneider et al. [8] provided early implementations, and subsequent improvements [9]–[12] have made UCs more efficient. Nevertheless, UCs also reveal the result to the server.

Oblivious RAM (ORAM), introduced by Pippenger, Goldreich, and Ostrovsky [13], [14], hides memory access patterns during computation. ORAM-based OTMs [15] provide a mechanism for secure arbitrary computations but typically require heavy interaction between parties, which is incompatible with our focus on low-bandwidth, decentralized computation.

Oblivious Turing Machines (OTMs) represent another line of work aimed at hiding both the function and the input. An early insecure proposal by Rass [16] was later corrected through the use of trusted hardware [17]. Goldwasser et al. [18], [19] introduced secure OTMs without trusted hardware, but the resulting constructions are complex and no public implementation is available. A more practical scheme was later introduced by Azogagh et al. [20], relying only on the LWE assumption and programmable bootstrapping. This scheme is the closest to our work in terms of non-interactivity and functional flexibility. Nevertheless, a key limitation of OTMs is their extremely high practical overhead: even simple computations require a vast number of rounds, making OTMs largely impractical for real-world use. This scheme is the only universal homomorphic function besides our proposal as far as we know.

Proposals such as Oblivious Virtual Machines or Zero-Knowledge Virtual Machines also do not satisfy our model because the former has shared data, function or result and the latter is used to do Zero-Knowledge proofs.

As conclusion of this overview, we provide a summarized comparison of the main primitives discussed. Table I outlines the guarantees each primitive offers regarding input privacy, function privacy, output privacy, non-interactivity, and support for arbitrary functions. This comparison highlights the specific positioning and advantages of our Oblivious Universal Function (OUF) scheme relative to existing methods.

Primitive	Input Privacy	Function Privacy	Output Privacy	Non-Interactive	Any Function	Reusable inputs
HE	✓	✗	✓	✓	✓	✓
PFE	✓	✓	∼	✗	✓	✗
FE	✓	✓	✗	✓	✗	✗
UC	✓	✓	✗	✓	✓	✗
ORAM	✓	∼	✓	✗	✓	✗
OTM	✓	✓	✓	✓	✓	✓
OUF	✓	✓	✓	✓	✓	✓

Table I: Feature comparison of cryptographic primitives. ✓: fully supported; ∼: partially; ✗: not supported.

II. Preliminaries

Having outlined the limitations of existing approaches and the need for better function confidentiality, we now introduce

the cryptographic tools and primitives underpinning our construction.

Our construction is based on Fully Homomorphic Encryption , which enables computations directly on encrypted data without decryption. Among various FHE schemes, we chose TFHE [21] for its Programmable Bootstrapping (PBS) procedure, which is central to our construction. TFHE is a lattice-based FHE scheme relying on the hardness of the (R)LWE problems. It operates over three ciphertext types (LWE, RLWE, and RGSW) supporting operations such as addition, absorption (scalar multiplication), and an external product for multiplying encrypted polynomials.

A key feature is Programmable Bootstrapping, which enables both ciphertext noise refreshing and the evaluation of arbitrary functions encoded in encrypted Look-Up Tables (LUTs). PBS allows evaluating a function f on an encrypted input $[\![x]\!]_{LWE}$ using an RLWE-encrypted LUT $[\![f]\!]_{RLWE}$, producing $[\![f(x)]\!]_{LWE}$ while maintaining privacy and ciphertext integrity.

At the core of PBS lies an operation called blind rotation, which homomorphically rotates the coefficients of an encrypted polynomial based on a secret value without revealing this value. More precisely, a blind rotation shifts the slots of an RLWE ciphertext according to the encrypted input $[\![x]\!]_{LWE}$, while preserving the encryption's secrecy. This mechanism allows selecting a specific entry inside an encrypted LUT without leaking any information about x to the server, enabling private and secure function evaluation.

TFHE's PBS enables evaluating any univariate function using encrypted LUTs. To evaluate a bivariate function $f(x, y)$, we use a composition of univariate look-ups, introduced as Blind Matrix Access (BMA) in [22].

BMA allows expressing a two-variable function f as a matrix. First, $[\![x]\!]_{LWE}$ is used to extract the x-th column from an encrypted LUT representing f, yielding an intermediate encrypted column. Then, PBS is used again to extract the y-th entry from that column using $[\![y]\!]_{LWE}$. This double-indirection process enables efficient and private evaluation of any bivariate function, a crucial component in our construction.

III. Our construction

In this work, we introduce the **Oblivious Universal Function (OUF)** scheme, a novel non-interactive approach to oblivious computation that ensures the confidentiality of both the data and the function being evaluated. Unlike prior constructions, our scheme preserves the same strong privacy guarantees for input, output, and function confidentiality as Oblivious Turing Machines while significantly improving practical performance and expressiveness. Existing OTM-based schemes are limited by rigid cryptographic parameters, constrained data sizes, and high computational costs. OUF addresses these limitations through a modular and scalable architecture, enabling the secure and efficient evaluation of arbitrary functions without interaction.

The key innovation of our construction lies in the integration of a virtual tape abstraction, inspired by the Turing machine model, with a fully encrypted instruction stream. In OUF, both the data and the function are provided as encrypted inputs to the server. The function is represented as a sequence of instructions encoded using lookup tables (LUTs), along with encrypted control logic that selects which sub-functions to apply and which tape positions to access. The tape serves as a dynamic memory structure where inputs, intermediate values, and outputs coexist and are updated homomorphically during computation.

A. General overview

The OUF scheme receives two main encrypted inputs: the input tape, which contains the data to be processed, and the function description, which guides the computation. The function description consists of three components: a set of encrypted sub-functions expressed as LUTs (either univariate or bivariate), a function selector array indicating which sub-function to apply at each step, and a movement selector array that determines which tape positions to read from and write to. These selectors are encrypted under the LWE scheme and used to control homomorphic operations such as blind rotation and programmable bootstrapping.

Figure 1 illustrates these components and how they interact during the execution of a single round. On the left, we show the encrypted tape, the sub-function LUTs, and the encrypted instruction arrays. On the right, we present a detailed example of a computation round, from input extraction to sub-function evaluation and tape update. The entire computation is structured as a sequence of such rounds. Each round executes a single operation defined by the encrypted function logic, and the server learns nothing beyond the total number of rounds, the upper bound on the number of sub-functions, and the tape length.

Each round proceeds as follows. The server first uses the encrypted movement selector to extract the required inputs from the tape. This is done by applying blind rotation operations to bring the target ciphertexts to index zero in the RLWE domain, followed by sample extraction to retrieve them as LWE ciphertexts. Once the input values are isolated, the server evaluates all available sub-functions on the input pair. This generates a list of candidate outputs, one for each sub-function.

To maintain obliviousness, the correct result is selected without revealing which function was applied. This is achieved through programmable bootstrapping, where the candidate outputs are packed into a lookup table and the appropriate result is extracted using the encrypted function selector. This PBS operation effectively performs a form of private information retrieval on encrypted data without interaction. The selected output is then written back to the tape at the position indicated

by another encrypted movement selector, again using PBS and blind rotation.

This process is repeated for each instruction in the encrypted function, with intermediate results stored on the tape and reused as needed in subsequent rounds. This non-interactive design allows all instructions and data to be transmitted in advance, and reused indefinitely across multiple evaluations without further communication.

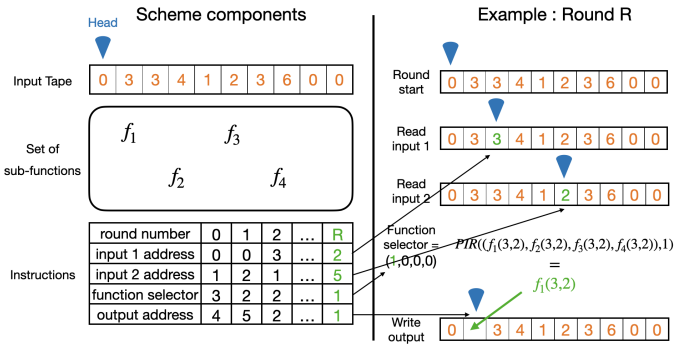

Figure 1: Overview of the OUF scheme. Left: encrypted components (tape, sub-functions, and instruction arrays). Right: one round of execution, showing encrypted input selection, function application, and tape update. Only the round number is in clear.

Figure 1 summarizes the scheme. On the left, it shows the encrypted components of the OUF scheme. On the right, it details how these elements interact during a single round of computation. The algorithm iterates through a series of such rounds to simulate a full function evaluation according to algorithm 1.

Algorithm 1 formalizes the logic of the OUF scheme. The procedure iterates over each instruction in the function selector array. For each instruction, it extracts two inputs from the tape using encrypted movement instructions, evaluates all sub-functions on these inputs, and selects the appropriate result using programmable bootstrapping based on the encrypted function selector. Finally, the selected result is written back to the tape at a new position also determined by an encrypted selector. All operations, including memory access and function selection, are performed homomorphically over encrypted data, ensuring complete obliviousness throughout the computation.

B. Evaluation of sub-functions and obliviousness

The set of sub-functions is fully customizable and defined by the client. It can include any bivariate function, from simple arithmetic operations to complex or randomized behaviors, without significantly affecting performance. This is made possible by the Blind Matrix Access (BMA) mechanism, which allows the secure indexing of precomputed function evaluations in an encrypted matrix. The server remains unaware of both the accessed indices and the stored values, ensuring the obliviousness of the computation regardless of the function's complexity.

Algorithm 1 Oblivious Universal Function

Input : T the tape, r the round number, F the set of sub-functions f_i, $F_{selector}$ the function selector array, M the head movement selector array

Ensure: Return the tape with the computation done

function $\texttt{OUF}(T, r, F, F_{selector}, M)$
  for $i = 0$ to $len(F_{selector})$ **do**
    $T \leftarrow$ **ChangeHeadPosition**$(T, M(3i))$
    $input_0 \leftarrow$ **ReadCellContent**(T)
    $T \leftarrow$ **ChangeHeadPosition**$(T, M(3i+1))$
    $input_1 \leftarrow$ **ReadCellContent**(T)
    $Storage \leftarrow []$ ▷ empty array
    for $j = 0$ to $len(F)$ **do**
      $Storage \leftarrow Storage || (f_j(input_0, input_1))$
    end for
    $Output \leftarrow$ **PIR**$(Storage, F_{selector}(i))$
    $T \leftarrow$ **ChangeHeadPosition**$(T, M(3i+2))$
    $T \leftarrow$ **WriteNewCellContent**$(T, Output)$
  end for
  return (T)
end function

The use of BMA also allows for dynamic and adaptive function definitions, enabling the client to modify or extend the function set without incurring substantial computational costs or compromising privacy. Although BMA may introduce overhead compared to directly applying simple functions, such as addition which is natively supported by the TFHE scheme. This trade-off can be managed depending on the sensitivity of the sub-function. In cases where the function itself does not reveal sensitive information, the client may choose to reveal it to optimize efficiency.

Obliviousness in this scheme operates on several levels. It first guarantees data privacy, ensuring that the client's inputs remain encrypted and undisclosed. It then preserves data access privacy, preventing the server from inferring which values are used or modified during computation. Furthermore, the scheme conceals the structure of the global function by hiding which sub-function is invoked at each step. Finally, sub-function privacy is maintained, meaning the server does not learn the content of the sub-functions themselves. This last level of obliviousness can be adjusted by the client, who may choose to expose or hide specific operations depending on their privacy requirements.

IV. Construction of an Universal Set of Sub-Functions

We now turn to the construction of sets of sub-functions that are expressive enough to support general-purpose computations up to Turing completeness. A key challenge is to keep these sets as small as possible to reduce computational

overhead, while maintaining the ability to evaluate complex functions in a limited number of rounds. This trade-off must be carefully considered by the client depending on the application.

A. Proof of Turing Completeness

Firstly, we can establish that this scheme is Turing complete by constructing a set of sub-functions capable of simulating a Turing Machine (TM). Simulation of a TM can be reduced to the simulation of a single TM round, as each round is structurally identical to any other round and the execution of an TM proceeds as a sequence of these rounds.

Each round of a TM depends on two variables: the machine's current state and the value at the position of the reading head. Using these two inputs, a round of the Turing Machine allows the server to obliviously compute three outcomes: (1) the symbol to be written at the current head position, replacing the existing value; (2) the direction in which the head will move (left or right); and (3) the next state of the machine.

Algorithm 2 Round of a TM

Input : T the tape, S the current state, H the head position and I the instruction table as a tensor such as $I = (I.write, I.move, I.state)$

Ensure: Return the tape with one more step

1: **function** $\textsc{Step}(T, S, H, I)$
2: $cellContent \leftarrow$ **ReadCellContent**(T)
3: $T \leftarrow$ **UpdateCellContent**$(T, cellContent, S, I.write)$
4: $H \leftarrow$ **ChangeHeadPos**$(H, cellContent, state, I.move)$
5: $S \leftarrow$ **GetNewState**$(cellContent, S, I.state)$
6: **return** (T, S, H)
7: **end function**

Algorithm 2 represents the algorithm to compute one round of a Turing Machine. It takes the current tape, state, head position and instructions as inputs and returns the updated tape, state and head position after one round.

A Turing Machine construction has several component : the tape, the head position, a state and the instructions which can be considered as 3 bivariate functions. Contrary to a TM, our construction can only read information from its tape which means that all necessary information besides functions, including the machine's state and the next head movement, must be stored on the input tape. An other issue is that a Turing Machine has a data-dependent access pattern whereas our construction is data-independent. This means that we cannot use a stored head position to access said position and that our data access is totally predetermined and would be the same for all Turing Machines.
To circumvent the first of these problems, we are going to reserve some slots of our input tape to store the Turing Machine elements that are outside the tape on a standard Turing Machine the following way : we designate the leftmost slot of the input tape to store the current step, the second slot to store the value under the reading head, and the third slot to store the movement of the head, represented by 1 or -1 (indicating a shift one slot to the right or one slot to the left). The fourth and fifth slots serve as temporary memory for moving the head, which helps us navigate the data-independent access pattern required by our scheme. The tape can be seen in figure 4.

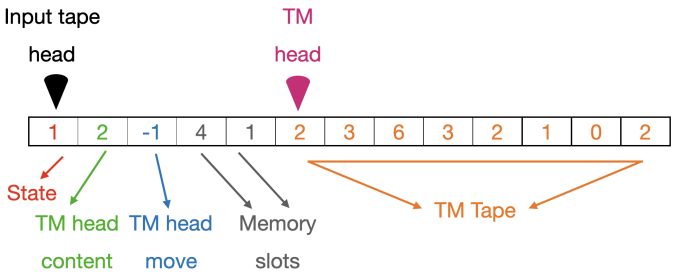

Figure 2: Tape layout for Turing Machine simulation. Cells 1–3 store control values (step count, cell content, head movement); cells 4–5 serve as memory. The remaining cells hold the actual TM tape. The head is shown for illustration only.

To solve the second problem, we will keep the head fixed in position and rotate the TM tape beneath it. This approach provides straightforward read/write access to the cell under the head. However, rotating the tape with a fixed access pattern requires sub-functions to shift each cell of the TM tape either one position to the left or one position to the right. Each round of our simulated TM begins by reading the element under the head and storing it in its designated slot. Based on the current state and the value read, the scheme then writes a new value into the cell under the head. Next, the movement of the head for the current round is computed and stored. The following step, which is the most complex part of the simulation, involves simulating a head movement. This requires simulating a trivariate function that combines the head's current position, the direction of movement, and the values on the TM tape. Finally, the machine's state is updated for the next round using a lookup table based on the current state and the read value. All these steps can be summed up as shown in Figure 3.

These sub-functions collectively enable to simulate an entire round of a Turing Machine and, by extension, a complete TM. This establishes that the scheme is Turing complete, despite its fixed access pattern and limitations to bivariate sub-functions.

B. Finding interesting sets of sub-functions for general computation

Our last set of sub-functions demonstrates the Turing completeness of the OUF scheme. However, this approach is impractical and inefficient for evaluating custom oblivious functions, as it simulates a system that, in turn, simulates the function rather than evaluating it directly. In this section, we explore how to design more efficient sets of sub-functions that

Instructions	Read under the head	Write under the head	Get the tape rotation	Rotating the tape	Updating the state
round number	0	1	2	k	k+1
input 1 address	5	0	0	...	0
input 2 address	5	1	1	...	1
function selector	0	1	2	...	7
output address	1	5	2	...	0

Figure 3: Instruction breakdown for TM simulation. Each column encodes an instruction type. Most steps take one round, except tape rotations, which require $14(n+1)$ rounds for n tape cells.

leverage the strengths of the OUF scheme to build custom oblivious functions.

Choosing a sub-function set involves a key trade-off: each added sub-function increases the cost of a round, but may also reduce the total number of rounds needed to evaluate the target function. The client must assess the utility of each sub-function and decide whether it's worth including. Too many specific sub-functions can hinder performance, while relying solely on basic ones may require significantly more rounds. In practice, a middle ground mixing simple and specialized functions often provides the best performance.

We focus here on the simple set Addition, Multiplication, denoted +, *. This set is compelling because it enables oblivious evaluation of any expression composed of additions and multiplications. The global functions expressible using this set correspond to multivariate polynomials over the ring $\mathbb{Z}_q[X]/(X^q+1)$, where q is the plaintext modulus. This illustrates how a small set of sub-functions can simulate an entire class of functions. Adding Subtraction, Division extends this capability further, allowing for efficient inverse operations and enabling functions like the mean in just two rounds.

Ultimately, OUF allows for domain-specific, privacy-preserving computation through carefully chosen sub-functions, without revealing the structure of the function itself. We illustrate this with two examples in the following sub-sections.

C. Oblivious Bubble Sort

Bubble Sort is a classical comparison sorting algorithm that iteratively sorts a list by comparing adjacent elements and swapping them if they are not in the desired order. Bubble Sort is described in Algorithm 3 and sorts elements by comparing each element with the previously sorted elements and placing it in the correct position. To implement an oblivious version of Bubble Sort, it is necessary to be able to compare two elements and either swap them if required or simulate a swap otherwise to stay oblivious. While a comparison can be achieved within a single computational round, swapping elements is more complex and necessitates multiple sub-functions.We introduce Algorithm 4 for obliviously swapping elements between cells and Algorithm 6 for performing the complete oblivious Bubble Sort process. Algorithm 4, also named Oblivious Compare and Swap (OCS),obliviously swaps elements between cells in 3 rounds and with only 2 very simple bivariate functions : Min and Max. The number of memory cells is reduced to its minimum to leave as much tape space as possible for the array to sort. This design illustrates the importance of choosing a domain specific set of sub-functions. Indeed, if the function was doing another task than sorting but needed an OCS, the OCS might be designed differently to minimize the number of sub-functions of the overall function.

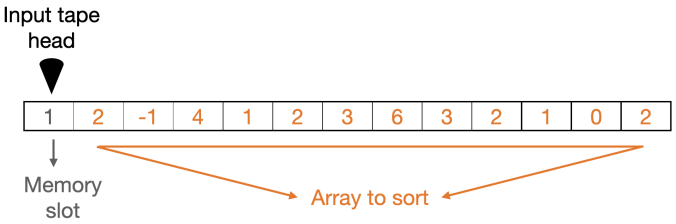

Figure 4: Tape layout for oblivious Bubble Sort. The first cell stores intermediate values used in the Oblivious Compare-and-Swap operation (Algorithm 4); the rest holds the array to be sorted.

Algorithm 3 Bubble Sort

Input : T the array of size n
Ensure: Sort T

```
function BUBBLE SORT(T)
    for i = 0 to n − 1 do
        Swap ← False
        for j = 0 to n − i − 1 do
            if T[j] > T[j + 1] then
                Temp ← T[j]
                T[j] ← T[j + 1]
                T[j] ← Temp
                Swap ← True
            end if
        end for
        if Swap = False then
            Break
        end if
    end for
    return T
end function
```

Algorithm 6 implements a full Oblivious Bubble Sort by repeatedly applying well-chosen Compare-and-Swap operations. Unlike the classical version, the oblivious variant cannot terminate early when the array is already sorted, as doing so would leak data-dependent control flow, which is not possible using homomorphic encryption. Instead, it has to execute the worst-case number of iterations which would still be the case for a non-oblivious homomorphic implementation. We evaluate the performance of this implementation in Section V.

Algorithm 5 Only two sub-functions are used to perform an *Oblivious Compare and Swap*: `Min` and `Max`. The objective of `Max` is to store in M the largest of the two elements, then the first `Min` overwrites B with the smallest element. The second `Min` acts as a copy function, as it is univariate and copies M into A. This approach reduces the number of sub-functions from three to two.

Input : 1 memory cell M and the 2 cells A and B
Ensure: Return the tape T with A and B swapped
1: **function** DOMAIN SPECIFIC OBLIVIOUS COMPARE AND SWAP(M, A, B)
2: $M \leftarrow Max(A, B)$
3: $B \leftarrow Min(A, B)$
4: $A \leftarrow Min(M, M)$
5: **return** T
6: **end function**

Algorithm 6 Oblivious Bubble Sort

Input : T the array of size n
Ensure: Obliviously sort T
1: **function** OBLIVIOUS BUBBLE SORT(T)
2: **for** $i = 0$ to $n - 1$ **do**
3: **for** $j = i + 1$ to 0 **do**
4: OCS($T, j, j - 1$)
5: **end for**
6: **end for**
7: **return** T
8: **end function**

V. RESULTS AND DISCUSSION

The Bubble Sort example illustrates the flexibility of the OUF model in implementing non-trivial functions. We now explore how performance is influenced by the choice of sub-functions, in particular, the trade-offs between privacy and computational cost before presenting empirical benchmarks of our implementation.

A. On the set of sub-functions

The sub-functions can be encoded under two models. The first option is to let some sub-functions public. If some sub-functions are public, several homomorphic optimizations can be used such as Multi-LUT evaluation or clear absorption of the result. The last case is obliviousness where the server learns nothing about the sub-function but no optimizations could be made.

We want to note that if the set of sub-functions is generic enough, it might not be worth privacy-wise to hide its content and that some specialized sub-functions might be done in several rounds using less advanced sub-functions. For example, the mean function could be done by first adding the numerator operands and by then dividing this result by two. Notice that any sub-function or global function is reusable as many times as needed by the client with different inputs. The same is true for all the inputs; they can be reused with any global function as much as the client wants. This is an important feature as most constructions of universal simulator intertwine the encryption of the input and the function, making both the function and the input one time use. This reusability is particularly important for clients with large volumes of computations, as transmitting only once a function greatly reduces communication cost.

An essential consideration in the design of functions is the architecture of the tape, which determines how data is organized and managed during computation. This architecture is particularly important for handling intermediary computations, which must be temporarily stored on the tape. In our examples, the initial slots of the tape are reserved to facilitate these operations, ensuring computational efficiency. The tape cells are broadly categorized into two types: input cells, which hold the primary data and final outputs, and memory cells, which store intermediate states and modifications of the input tape during processing. Selecting an appropriate tape architecture involves balancing efficient computation with the availability of sufficient input cells to enable meaningful and complex computations.

B. Results

We implemented our scheme in Rust using the tfhe.rs library. The source code is publicly available on GitHub and all experiments were conducted on a machine equipped with an AMD Ryzen 7 3800X CPU and 16GB of RAM.

Modulus	2 functions	4 functions	8 functions	16 functions
8	633	1213	2509	-
16	1737	3421	6757	13542
32	7992	15979	31834	63607

Table II: Round computation time (ms) for different plaintext moduli and numbers of sub-functions.

Table II reports the time required to perform a single round of computation in our scheme, for varying numbers of sub-functions and plaintext modulus sizes. Since each sub-function is evaluated independently using a Blind Matrix Access (BMA), the cost per round scales linearly with the number of sub-functions. Increasing the modulus also significantly impacts performance as it expands both the representable range and the tape length. Each modulus size represents a cryptographic parameter that affects performance, the higher the slower. These measurements can be used to estimate the total computation time for a given function by multiplying the round cost by the number of rounds required.

Table III illustrates a practical use case: obliviously sorting encrypted values using our Bubble Sort algorithm. Each comparison and simulated swap requires 3 computation rounds,

	$k=2$	$k=3$
Algorithm 4	1.2	3.7
OTM [20]	188	761

Table III: Sorting time (s) to obliviously sort k elements using our Compare-and-Swap (Algorithm 4) and the best known OTM implementation [20].

resulting in a total of $\frac{3}{2}(k^2 - k)$ rounds to sort k elements. While this leads to a high round count, the goal is not efficiency but full function confidentiality: the server only observes the number of rounds and sub-functions used, without learning that the computation performs sorting.

The small values of k in this benchmark are imposed by the limitations of the comparison target, Oblivious Turing Machines (OTMs), which require larger functions, tapes, and cryptographic parameters. Despite this, the runtime of OUF is significantly lower than that of OTMs, making it a practical alternative within the same security model.

Table IV extends this benchmark to larger input sizes, providing a broader view of OUF's scalability and runtime behavior.

These results demonstrate the feasibility and comparative efficiency of OUF, especially when benchmarked against existing oblivious computing frameworks.

	Modulus	$k=5$	$k=10$	$k=20$
Algorithm 4	8	19.1	-	-
	16	53.4	213	-
	32	226	1062	4568

Table IV: Sorting time (s) for k elements using Compare-and-Swap (Algorithm 4). The modulus limits value range and array size; – indicates insufficient capacity.

VI. Conclusion

In this paper, we presented a universal oblivious computing scheme that efficiently evaluates arbitrary functions by composing specialized sub-functions. Supporting rich bivariate operations, it ensures strong privacy by concealing not only inputs and outputs but also the function logic itself, a crucial property for real-world applications such as secure data analytics, confidential algorithm deployment, and outsourced computation.

The proposed framework combines expressiveness, reusability, and non-interactivity, making it suitable for practical deployment scenarios where sensitive computations must run untrusted environments. Future improvements include expanding plaintext capacity for higher throughput, reducing overhead by enabling encrypted selection of a single sub-function per round, and adding support for higher-arity logic and scalable encoding. With these optimizations, OUF can serve as a foundation for general-purpose, privacy-preserving computation in cloud services, collaborative analytics platforms, and secure machine-learning pipelines.

References

[1] J. Katz and L. Malka, "Constant-round private function evaluation with linear complexity," Cryptology ePrint Archive, Report 2010/528, 2010. [Online]. Available: https://eprint.iacr.org/2010/528

[2] P. Mohassel and S. Sadeghian, "How to hide circuits in MPC: An efficient framework for private function evaluation," Cryptology ePrint Archive, Report 2013/137, 2013. [Online]. Available: https://eprint.iacr.org/2013/137

[3] P. Mohassel, S. Sadeghian, and N. P. Smart, "Actively secure private function evaluation," Cryptology ePrint Archive, Report 2014/102, 2014. [Online]. Available: https://eprint.iacr.org/2014/102

[4] O. Bicer, M. A. Bingol, and M. S. Kiraz, "Highly efficient and reusable private function evaluation with linear complexity," Cryptology ePrint Archive, Report 2018/515, 2018. [Online]. Available: https://eprint.iacr.org/2018/515

[5] Z. Brakerski and G. Segev, "Function-private functional encryption in the private-key setting," Cryptology ePrint Archive, Report 2014/550, 2014. [Online]. Available: https://eprint.iacr.org/2014/550

[6] M. Abdalla, D. Catalano, D. Fiore, R. Gay, and B. Ursu, "Multi-input functional encryption for inner products: Function-hiding realizations and constructions without pairings," in *CRYPTO 2018, Part I*, ser. LNCS, H. Shacham and A. Boldyreva, Eds., vol. 10991. Springer, Cham, Aug. 2018, pp. 597–627.

[7] L. G. Valiant, "Universal circuits (preliminary report)," in *Proceedings of the eighth annual ACM symposium on Theory of computing*, 1976, pp. 196–203.

[8] V. Kolesnikov and T. Schneider, "A practical universal circuit construction and secure evaluation of private functions," in *FC 2008*, ser. LNCS, G. Tsudik, Ed., vol. 5143. Springer, Berlin, Heidelberg, Jan. 2008, pp. 83–97.

[9] A.-R. Sadeghi and T. Schneider, "Generalized universal circuits for secure evaluation of private functions with application to data classification," Cryptology ePrint Archive, Report 2008/453, 2008. [Online]. Available: https://eprint.iacr.org/2008/453

[10] Á. Kiss and T. Schneider, "Valiant's universal circuit is practical," in *EUROCRYPT 2016, Part I*, ser. LNCS, M. Fischlin and J.-S. Coron, Eds., vol. 9665. Springer, Berlin, Heidelberg, May 2016, pp. 699–728.

[11] H. Liu, Y. Yu, S. Zhao, J. Zhang, W. Liu, and Z. Hu, "Pushing the limits of valiant's universal circuits: Simpler, tighter and more compact," in *CRYPTO 2021, Part II*, ser. LNCS, T. Malkin and C. Peikert, Eds., vol. 12826. Virtual Event: Springer, Cham, Aug. 2021, pp. 365–394.

[12] Y. Disser, D. Günther, T. Schneider, M. Stillger, A. Wigandt, and H. Yalame, "Breaking the size barrier: Universal circuits meet lookup tables," in *ASIACRYPT 2023, Part I*, ser. LNCS, J. Guo and R. Steinfeld, Eds., vol. 14438. Springer, Singapore, Dec. 2023, pp. 3–37.

[13] O. Goldreich, "Towards a theory of software protection and simulation by oblivious RAMs," in *19th ACM STOC*, A. Aho, Ed. ACM Press, May 1987, pp. 182–194.

[14] R. Ostrovsky, "Efficient computation on oblivious RAMs," in *22nd ACM STOC*. ACM Press, May 1990, pp. 514–523.

[15] T. Moataz, E.-O. Blass, and T. Mayberry, "Chf-oram: a constant communication oram without homomorphic encryption," *Cryptology ePrint Archive*, 2015.

[16] S. Rass, "Blind turing-machines: Arbitrary private computations from group homomorphic encryption," *arXiv preprint arXiv:1312.3146*, 2013.

[17] S. Rass, P. Schartner, and M. Brodbeck, "Private function evaluation by local two-party computation," *EURASIP Journal on Information Security*, vol. 2015, pp. 1–11, 2015.

[18] S. Goldwasser, Y. Kalai, R. A. Popa, V. Vaikuntanathan, and N. Zeldovich, "How to run Turing machines on encrypted data," Cryptology ePrint Archive, Report 2013/229, 2013. [Online]. Available: https://eprint.iacr.org/2013/229

[19] S. Goldwasser, Y. T. Kalai, R. A. Popa, V. Vaikuntanathan, and N. Zeldovich, "Overcoming the worst-case curse for cryptographic constructions." *IACR Cryptol. ePrint Arch.*, vol. 2013, p. 229, 2013.

[20] S. Azogagh, V. Delfour, and M.-O. Killijian, "Oblivious turing machine," in *2024 19th European Dependable Computing Conference (EDCC)*. IEEE, 2024, pp. 17–24.

[21] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, "Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds," Cryptology ePrint Archive, Report 2016/870, 2016. [Online]. Available: https://eprint.iacr.org/2016/870

[22] S. Azogagh, V. Deflour, and M.-O. Killijian, "Oblivious turing machine," Cryptology ePrint Archive, Report 2023/1643, 2023. [Online]. Available: https://eprint.iacr.org/2023/1643